Faculty of Engineering & Technology

Computer Science Department

Data Base Systems – COMP333

Course Project

Final Project Report

Prepared by:

Name: Lara Daifallah **Number:** 1230239

Name: Shatha Abualrub **Number:** 1231279

Instructor: Dr. Bassem Sayrafi

Section: 2

Date: 14/6/2026

Table of Contents

Project Description

This project is centered on developing a database application for **Health Plus**, a warehouse based in Abu Qash that distributes pharmaceuticals and personal care products to pharmacies and clinics in the surrounding area. The warehouse stocks a broad range of items, covering pharmaceuticals, skincare products, baby care essentials, shampoos, and various other health-related supplies.

The system is built to support the warehouse's everyday operations, including inventory tracking, supplier management, and recording of sales transactions. All data concerning products, categories, suppliers, customers, and financial activity will be stored in a well-organized structure, enabling efficient querying and the production of detailed reports whenever needed.

Project Scope

This project covers the design and development of a database system that handles the core day-to-day operations of the Health Plus warehouse.

The system will:

- Track product information: name, category, available quantity, and expiration date
- Manage supplier and customer records
- Record purchase transactions from suppliers
- Record sales transactions to pharmacies and clinics
- Monitor inventory levels and product availability
- Provide query-based reports to support management decisions

Note: features like online ordering or automated payment gateways fall outside the boundaries of this project; our focus remains entirely on the warehouse's internal operations.

Client Information

The client for this project is Health Plus, a commercial warehouse that focuses on supplying pharmaceuticals and personal care products to pharmacies and clinics.

Health Plus serves as a link between suppliers and healthcare providers, buying goods from several suppliers and distributing them to pharmacies and clinics across different regions. The warehouse manages a wide variety of products, from pharmaceuticals and skincare lines to other health-related supplies.

Contact Information:

- **Name:** Health Plus .
- **Address:** AbuQash Main St., Ramallah, Palestine
- **Phone:** +970 (0)2 281 1034
- **Email:** mailto:info@healthplus.ps
- **Website:** https://healthplus.ps/

Entities Description

The entities listed below were identified based on how Health Plus actually runs its daily operations. Each one corresponds to a concrete, real-world element that the system is expected to track and manage.

Warehouse

The Warehouse entity represents the physical storage facility operated by Health Plus. It stores information such as *warehouse ID*, *warehouse name*, *address*, *city*, *storage capacity*, and *phone number*. Although Health Plus currently operates a single warehouse, the entity was included to support future expansion and to properly separate warehouse information from product information.

Product

The Product entity captures the general information of each stock-keeping unit (SKU) managed by the warehouse. It stores the *product name*, *description*, *price*, *reorder level*, and *category*. Inventory-related information such as quantity in stock, expiration date, and storage location is maintained separately through the Batch entity. Every product is associated with exactly one category and can be sourced from one or more suppliers.

Batch

The Batch entity represents the actual inventory stored in the warehouse. Each batch belongs to one product and one warehouse and stores information such as *quantity in stock*, *expiration date*, and *storage location*. This design allows the same product to exist in multiple batches with different expiration dates and quantities, which is especially important for pharmaceutical products.

Category

The Category entity groups products into meaningful classifications, such as pharmaceuticals, skincare, baby care, shampoos, and personal care items. Each category is identified by a unique

ID, a name, and a short description. While a category can hold many products, each product is assigned to only one category.

Supplier

The Supplier entity holds information about the vendors that Health Plus purchases its products from. This includes the *supplier name, contact person, phone number, email address, physical address, and city*. A supplier can provide multiple products, and a product can be sourced from more than one supplier.

Client

The Client entity covers the pharmacies and clinics that buy products from the warehouse. For each client, the system stores their *name, type (pharmacy or clinic), phone number, location details, and assigned credit limit*. A single client can place any number of sales orders throughout their relationship with the warehouse.

Employee

The Employee entity stores information about the staff members who keep the warehouse running. It captures each *employee's first and last name, job role (Warehouse Manager, Sales Officer, or Procurement Officer)*, hire date, phone number, salary, and the warehouse they are assigned to. Employees are linked to the purchase and sale orders they handle, and each employee is assigned to one warehouse.

UserAccount

The UserAccount entity stores login credentials and access type for system users. Each account has a unique *UserID, password, and role*. A user account may belong to an employee, a client, or represent an admin account. Employee accounts carry an EmployeeID foreign key linking them to an Employee record. Client accounts carry a ClientID foreign key linking them to a Client record. Admin accounts have both foreign keys set to null, as they represent system-level access rather than a specific person.

PurchaseOrder

The Purchase Order entity captures every order that Health Plus sends to a supplier to restock inventory. Each record includes the purchase order number, the relevant supplier, the employee who created the order, the order date, the expected delivery date, the current status (pending, delivered, or canceled), and the total value of the order.

PurchaseOrderItem

This entity breaks down a purchase order into its individual line items. Each record points to a *specific product* and *captures the quantity ordered, the agreed unit cost, and the amount* actually received when the delivery arrives. This level of detail makes it straightforward to track and reconcile partial deliveries.

SaleOrder

The Sale Order entity captures every customer order placed with the warehouse. It records the *Sale Order ID*, the client who placed it, the employee who handled it, *the order and expected delivery dates, the current order status, the total value*, and whether *payment* has been made in full, partially, or not at all.

SaleOrderItem

This entity holds the line-by-line breakdown of each sale order. Every item includes the batch selected, the quantity requested, the unit price at the time of sale, and any discounts that were applied. Referencing batches instead of products allows the system to identify the exact inventory batch used in each sale and maintain product traceability.

InventoryTransaction

The Inventory Transaction entity keeps a full audit trail of all stock movements in the warehouse. Each record identifies the batch involved*, the type of transaction (receipt, dispatch, or adjustment), the quantity affected*, the *exact date, a reference* to the related purchase or sale order, and the employee responsible for carrying out the transaction.

Payment

The Payment entity tracks all financial transactions tied to sales and purchase orders alike. It stores *the payment date, the amount paid, the method used, and the direction of the payment,* whether money was coming in from a client or going out to a supplier. This setup makes it easy to manage and monitor full payments, partial payments, and installment arrangements on both sides.

Note on Phone Number: Phone number is technically a multi-valued attribute. In a full ER design this would normally be pulled out into a separate table. But in our project we kept it as a single column because in practice none of the employees, clients, or suppliers in our system have more than one phone number on record. So we dealed with it as single valued attribute.

Relationships

None of the entities above stands alone - they are linked to one another in ways that mirror how the warehouse actually operates. The relationships described below capture these connections, along with their cardinality, participation constraints, and any attributes that logically belong to the relationship rather than to the individual entities involved.

- **Product - Category (M-1):**
  Every product must be assigned to a category, which means a product's participation in this relationship is total. A category, on the other hand, can hold many products but can also exist in the system before any product has been added to it, making its participation partial.
- **Product - Batch (1-M)**

  Each batch belongs to exactly one product, while a product can have many batches. Batch participation is total because a batch cannot exist without a product. Product participation is partial because a product may exist before inventory is received.

- **Warehouse - Batch (1-M)**

  Each batch is stored in exactly one warehouse, while a warehouse can contain many batches. Batch participation is total because every batch must be stored in a warehouse. Warehouse participation is partial because a warehouse may exist before inventory is stored in it.

- **Warehouse - Employee (1-M):**

Each employee is assigned to one warehouse, while a warehouse can have many employees working in it. Employee participation is partial because an employee record may exist before a warehouse assignment is set. Warehouse participation is partial because a warehouse can exist before any employees are assigned to it. This relationship is represented by the WarehouseID foreign key stored in the Employee table.

- **Employee - UserAccount (1 - N):**

An employee may have one user account used to log into the employee portal. A user account with the Employee role must be linked to exactly one employee. Employee participation is partial because an employee record may exist before an account is created.

- **Client - UserAccount (1 - N):**

A client may have one user account used to log into the client portal and place orders. A user account with the Client role must be linked to exactly one client. Client participation is partial because a client can be registered before receiving login access.

- **Supplier -Product (M-N):**

  A supplier can offer multiple products, and a product can come from several different suppliers. Participation is partial on both sides. An important feature of this relationship is the UnitCost attribute, which records the price agreed upon between a specific supplier and a specific product. This attribute belongs to the relationship rather than to either entity individually, since the same product may be priced differently by different suppliers.

- **Supplier - PurchaseOrder (1-M):**

  Each purchase order is placed with a single supplier, while a supplier can receive many purchase orders over time. Since a purchase order cannot exist without a supplier, it has total participation in this relationship. A supplier, however, may be registered in the system before any orders are placed with them, giving it partial participation.

- **PurchaseOrder - PurchaseOrderItem (1-M):**

  Each purchase order item belongs to exactly one purchase order, and a purchase order can include many items. Since a purchase order must have at least one item to be meaningful, it has total participation. PurchaseOrderItem is a weak entity, meaning it has no independent primary key of its own, its identity is derived from its parent order through a composite key made up of the PONumber and POItemID.

- **PurchaseOrderItem - Product (M-1):**

  Every line item in a purchase order is tied to exactly one product, though the same product can appear across many order items over time. PurchaseOrderItem has total participation, since a line item must always reference a product. Product participation is partial, as a product can be listed in the catalog without ever having been ordered.

- **Client - SaleOrder (1-M):**
  Every sale order must be associated with a client, so SaleOrder has total participation in this relationship. Clients have partial participation, since a client can be added to the system before they have placed any orders.
- **SaleOrder - SaleOrderItem (1-M):**
  Much like the purchase order structure, SaleOrderItem is a weak entity that depends entirely on its parent SaleOrder to exist. An item has no meaning on its own — it is identified by combining the SaleOrderID from the parent order with its own partial key, the SaleItemID. A sale order must always contain at least one item, and each item belongs to exactly one sale order, giving both sides total participation in this relationship.
- **SaleOrderItem - Batch (M-1):**
  Each sale order item references exactly one batch, while a batch can appear in many sale order items over time. SaleOrderItem has total participation because every sold item must originate from a specific batch. Batch participation is partial because a batch may exist in inventory without having been sold yet.
- **Employee - PurchaseOrder (1-M):**
  Every purchase order must be linked to the employee who created it, so PurchaseOrder has total participation here. An employee may not have any purchase orders attached to their record yet, meaning their participation is partial. This relationship helps the system maintain accountability and keep track of each employee's workload.
- **Employee - SaleOrder (1-M):**
  Just as with purchase orders, every sale order must be assigned to the employee who processed it, so SaleOrder has total participation. An employee can handle many sale orders over time, but may also have none linked to them initially, making the Employee's participation partial.
- **Employee - InventoryTransaction (1-M):**
  The system does not allow anonymous inventory transactions,every transaction must be tied to the employee who carried it out, so InventoryTransaction has total participation. Employees have partial participation since not all employees will necessarily be involved in inventory transactions.

- **Batch - InventoryTransaction (1-M):**
  For each inventory transaction affects exactly one batch, while a batch can accumulate many transaction records over time. InventoryTransaction has total participation because every transaction must reference a batch. Batch participation is partial because a newly received batch may not yet have transaction records.
- **SaleOrder - Payment (1-M):**
  A sale order can be paid off through one or more payments, allowing the system to handle both partial and installment arrangements. SaleOrder has partial participation since some orders may remain unpaid for a period. Each payment, however, must belong to exactly one sale order, giving Payment total participation. This design makes it easy to track the outstanding balance for each order.
- **PurchaseOrder - Payment (1-M):**
  A purchase order can be settled in one or more outgoing payments to the supplier. PurchaseOrder has partial participation because an order may be placed before any payment is sent. Each outgoing payment references the purchase order it settles, and either the SaleOrderID or the PONumber will be set on a payment record, but not both. This arrangement allows the warehouse to track full, partial, and installment payments on both incoming and outgoing transactions.

Entity-Relationship Diagram (ERD)

The figure below presents the ER diagram for the Health Plus warehouse management system. The diagram follows standard Chen Notation, using rectangles for entities, diamonds for relationships, and ovals for attributes. Weak entities appear as double rectangles, and total participation is indicated by double lines connecting an entity to its relationship.

Figure 1: ER Diagram- Health Plus Warehouse Management System

ERD Description

The ERD illustrates fourteen entities and the relationships that tie them together.

Weak Entities

The diagram includes two weak entities, meaning two entities that do not have an independent primary key and rely on a parent entity for their identity:

- **PurchaseOrderItem** This is a weak entity that depends on its parent purchase order for its existence. A line item has no meaning on its own, outside of the order it belongs to. Its identifying relationship is "Contains," and it is identified by a composite key consisting of the PONumber (inherited from the parent) and the partial key POItemID.
- **SaleOrderItem** This is a weak entity that relies on a sale order for its identity. It cannot be uniquely identified without reference to its parent order. Its identifying relationship is "Has," and it uses a composite key made up of the SaleOrderID and the partial key SaleItemID.

Total Participation

Total participation (shown as double lines in the ERD) means that every instance of an entity must participate in the relationship. The following entities have total participation in the relationships indicated:

- Every **Product** must belong to a Category ("Classifies" relationship).
- Every **PurchaseOrder** must be directed to a Supplier ("Issues" relationship) and raised by an Employee ("Handles" relationship).
- Every **SaleOrder** must be associated with a Client ("Places" relationship) and processed by an Employee ("Handles" relationship).
- Every Batch must belong to a Product and be stored in a Warehouse.
  Every **InventoryTransaction** must reference a Batch ("Tracks" relationship) and be performed by an Employee ("Performs" relationship).

- Every **PurchaseOrderItem** must belong to a PurchaseOrder (Contains relationship), total participation because it's a weak entity.
- Every **PurchaseOrderItem** must reference a Product (OrdersProduct relationship).
- Every **SaleOrderItem** must belong to a SaleOrder (Has relationship), total participation because it's a weak entity.
- Every **SaleOrderItem** must reference a Batch (FromBatch relationship).

- Every **Payment** must reference either a SaleOrder (for incoming payments from clients) or a PurchaseOrder (for outgoing payments to suppliers), but not both at the same time.

Relationship Attributes

In our system, the "Supplies" relationship between a supplier and a product carries the attribute UnitCost, which reflects the price that a specific supplier charges for a specific product. Attaching this attribute to either entity alone would be inaccurate, since the same product can have a different cost depending on the supplier. In the diagram, this attribute is shown as a dotted oval attached to the "Supplies" relationship to make this distinction clear.

Technology to be Used

The system will be built using **Java** as the primary programming language, with **JavaFX** handling the design and implementation of the graphical user interface (GUI). Data storage and management will be handled by a **MySQL** database for storing and managing data. Together, these technologies provide a solid and practical foundation for a desktop-based warehouse management system.

Sample Queries

- **Inventory & Stock**

1. Retrieve each role with the number of employees in it, the minimum, maximum, and average salary, sorted by average salary descending.
2. Retrieve all warehouses located in Ramallah, along with the total number of product batches currently stored in each.
3. Retrieve the product name, category, quantity in stock, expiry date, and storage location for all batches currently stored in Health Plus Main Warehouse, sorted by expiry date ascending.
4. Retrieve all products in Health Plus Main Warehouse whose batch quantity has fallen below the product's reorder level, sorted by category name.
5. Retrieve the complete inventory transaction history for Panadol 500mg batch (1), showing transaction type, quantity changed, date, and the name of the employee who performed it, sorted by transaction date ascending.

- **Suppliers & Procurement**

6. Retrieve all products supplied by Al-Quds Pharma that are currently stocked in Health Plus Main Warehouse, including the agreed unit cost, batch quantity, expiry date, and storage location, sorted by expiry date ascending.

7. List all suppliers along with the total number of distinct products they supply and the total number of purchase orders issued to each, sorted by total purchase orders descending.
8. Retrieve all purchase orders issued in 2026, showing supplier name, order status, expected delivery date, and total order amount, sorted by expected delivery date ascending.
9. Find the total value of products received from each supplier in 2026, sorted by total value descending.
10. Retrieve all purchase orders with a status of Pending whose expected delivery date has already passed 1-6-2026, indicating potential supply delays, sorted by expected delivery date ascending.

- **Clients & Sales**

11. List the name, type, city, and total number of orders placed by each client, including clients who have not placed any orders yet, sorted by total orders descending.

12. Retrieve all sale orders placed by a Betunia Grand Pharmacy, showing order date, delivery date, status, total amount, and payment status, sorted by order date descending.
13. Find the top five best-selling products by total quantity sold in May 2026, along with the warehouse each batch originated from.

- **Employees**

14. List the full name, role, hire date, phone, and salary of all employees whose role contains the word Manager, Sales, or Officer, sorted by role then by hire date ascending.
15. Retrieve the full name, phone, and hire date of all Sales Officers, sorted by hire date ascending.

16. Find the employee who processed the highest number of sale orders in the current month, along with the total value of those orders.

- **Payments & Finance**

17. Retrieve all payments received from clients in June 2026, showing client name, payment date, amount paid, and payment method, sorted by payment date descending.

- **Complex Queries**

18. Calculate the total number of product batches per category at Health Plus Main Warehouse, along with the combined stock value per category, sorted by total stock value descending.
19. Calculate the total revenue generated per product category for 2026, sorted by revenue descending.
20. Calculate the total revenue generated per product category for 2026, sorted by revenue descending.
21. Identify all sale orders that are fully or partially unpaid and are more than 30 days old, showing client name, order date, total amount, and payment status, sorted by order date ascending.
22. Find the employee who processed the highest number of sale orders in January 2025, along with the total value of those orders.

Normalization to Third Normal Form (3NF)

Normalization was applied to the Health Plus database schema to reduce redundancy, avoid update anomalies, and ensure that each relation stores facts about one main entity or relationship. The final schema was checked against First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

- **First Normal Form (1NF)**

All relations in the database satisfy First Normal Form because every table has a primary key, and all attributes contain atomic values. There are no repeating groups or multi-valued attributes inside a single field. For example, products supplied by multiple suppliers are not stored as a list inside the Product table. Instead, the many-to-many relationship between Supplier and Product is represented using the SupplierProduct table.

- **Second Normal Form (2NF)**

The schema satisfies Second Normal Form because every non-key attribute depends on the whole primary key. Tables with single-attribute primary keys, such as Product, Category, Supplier, Client, Employee, Warehouse, Batch, PurchaseOrder, SaleOrder, InventoryTransaction, and Payment, automatically satisfy 2NF once they are in 1NF.

For tables with composite keys, such as SupplierProduct, PurchaseOrderItem, and SaleOrderItem, the non-key attributes depend on the full key and not only part of it. For example, in SupplierProduct, UnitCost depends on both SupplierID and ProductID because the cost is determined by the specific supplier-product pair.

- **Third Normal Form (3NF)**

The schema satisfies Third Normal Form because there are no transitive dependencies between non-key attributes. Each non-key attribute depends only on the primary key of its own table and not on another non-key attribute.

For example, CategoryName and Category Description are stored in the Category table instead of being repeated inside Product. Supplier details are stored only in Supplier and not repeated inside PurchaseOrder. Client details are stored only in Client and not repeated inside SaleOrder. Employee information is stored only in Employee and not repeated inside purchase orders, sale orders, or inventory transactions.

The Product table was also improved to satisfy normalization principles. Inventory-related attributes such as QtyInStock, ExpiryDate, and StorageLocation were removed from Product and placed in the Batch table. This is because these attributes describe a specific inventory batch, not the product itself. A single product can have many batches with different quantities, expiry dates, and storage locations. Separating Batch from Product reduces redundancy and avoids incorrect updates when the same product exists in multiple batches.

The Warehouse table was also introduced to separate warehouse information from inventory records. Health Plus currently operates three warehouses across different cities, and this design supports them all without changing the structure of the Product table. The WarehouseID foreign key in the Employee table captures which warehouse each employee is assigned to. This is a direct dependency on the Employee primary key and does not introduce any transitive dependency, so the table remains in 3NF.

Therefore, the final database schema is in 3NF because each relation represents one entity or relationship, all attributes are atomic, all non-key attributes depend on the whole key, and no non-key attribute depends on another non-key attribute.

- **Final Normalized Relations**

Category(CategoryID, Name, Description)

Product(ProductID, ProductName, Description, UnitPrice, ReorderLevel, CategoryID)

Warehouse(WarehouseID, WarehouseName, Address, City, Phone, Capacity)

Batch(BatchID, ProductID, WarehouseID, QtyInStock, ExpiryDate, StorageLocation)

Supplier(SupplierID, SupplierName, ContactPerson, Phone, Email, City)

SupplierProduct(SupplierID, ProductID, UnitCost)

Client(ClientID, ClientName, ClientType, Phone, City, CreditLimit)

Employee(EmployeeID, FirstName, LastName, Role, HireDate, Phone, Salary, WarehouseID)

PurchaseOrder(PONumber, SupplierID, EmployeeID, OrderDate, ExpDeliveryDate, Status, TotalAmount)

PurchaseOrderItem(POItemID, PONumber, ProductID, QtyOrdered, UnitCost, QtyReceived)

SaleOrder(SaleOrderID, ClientID, EmployeeID, OrderDate, DeliveryDate, Status, TotalAmount, PaymentStatus)

SaleOrderItem(SaleItemID, SaleOrderID, BatchID, QtyOrdered, UnitPrice, Discount)

InventoryTransaction(TransactionID, BatchID, EmployeeID, TxnType, Quantity, TxnDate, ReferenceID)

Payment(PaymentID, SaleOrderID, PONumber, PaymentDate, Amount, PaymentMethod, Direction)

UserAccount(UserID, Password, Role, EmployeeID, ClientID)